



MINISTÉRIO DA EDUCAÇÃO
SECRETARIA DE EDUCAÇÃO PROFISSIONAL E TECNOLÓGICA
INSTITUTO FEDERAL DO NORTE DE MINAS GERAIS – IFNMG
Criado pela Lei no.11.892, de 29/12/2008
Campus Montes Claros



SARAH EMANUELLE ALVES LINO
JOÃO KENNEDY SOUZA SOARES

RELATÓRIO : TRABALHO PRÁTICO 3
OpenGL 2D - Implementação de um Jogo Clássico

Relatório apresentado ao Professor Wagner Ferreira de Barros como parte das exigências de avaliação da disciplina de Computação Gráfica do Curso de Bacharelado em Ciência da Computação do Instituto Federal do Norte de Minas Gerais, Campus Montes Claros.

MONTES CLAROS - MG

2025

Relatório de Desenvolvimento: OpenGL 2D - Implementação de um Jogo Clássico

A Computação Gráfica é a área da ciência da computação focada na geração, processamento e visualização de imagens através de computadores. Uma de suas ferramentas fundamentais é a API OpenGL (Open Graphics Library), uma interface de programação de aplicações multiplataforma utilizada para renderizar gráficos 2D e 3D.

Diferente de bibliotecas de alto nível, o OpenGL fornece um conjunto de comandos de baixo nível que permitem ao desenvolvedor controlar diretamente o hardware gráfico (GPU). O modelo de renderização do OpenGL é baseado no envio de primitivas geométricas (pontos, linhas e polígonos) para um pipeline gráfico, que as processa e as converte em pixels na tela. Este trabalho foca no uso desses conceitos fundamentais para a criação de uma cena 2D estática com elementos de interação.

2. Visão Geral e Objetivos do Projeto

O objetivo central deste projeto foi desenvolver uma aplicação gráfica utilizando a linguagem C++ e a biblioteca OpenGL para desenhar uma cena de um jogo em 2D e implementar mecânicas de interação com o usuário através do teclado e do mouse.

As funcionalidades principais implementadas incluem:

- Renderização de uma cena 2D composta por múltiplos objetos geométricos utilizando primitivas do OpenGL como *GL_POLYGON*, *GL_QUADS* e *GL_LINES*.
- Configuração de um sistema de coordenadas 2D usando *gluOrtho2D*.
- Implementação de um loop de atualização e renderização a aproximadamente 60 FPS, controlado via *glutTimerFunc*.
- Criação de classes para gerenciar a lógica e o estado das entidades: Player, Alien, AlienSwarm, e Projectile
- Controle do fluxo do jogo através de estados definidos (MENU, PLAYING, GAME_OVER, WIN).

- Captura de entradas do teclado (movimentação, tiro) e exibição de informações (Score, Lives) usando texto bitmap do GLUT
- Integração de música de fundo e efeitos sonoros com a biblioteca SDL2_mixer, adaptando-se ao estado do jogo.

3. Desenvolvimento e Tecnologias

Para a construção do projeto, foram utilizadas ferramentas, com foco na integração de bibliotecas clássicas do C++ para gráfico e multimídia.

- Linguagem de Programação: C++11 foi a linguagem principal, utilizando sua capacidade de programação orientada a objetos.
- API Gráfica: OpenGL foi utilizado para todos os comandos de desenho, configuração de matrizes e renderização
- Entradas e Janelas: Foi empregado o GLUT (freeglut) para a criação da janela, gerenciamento de contexto OpenGL e registro das funções de callback de teclado e temporizador
- Áudio: As bibliotecas SDL2 e SDL2_mixer foram usadas para carregar e reproduzir arquivos de áudio .mp3 (músicas) e .wav (efeitos sonoros).
- Compilação: O projeto utiliza o g++ e make para a compilação e linkagem das dependências.

A estrutura do código foi modularizada em classes, conforme os arquivos de cabeçalho (arquivos .h) e implementações (arquivos .cpp). A classe Game centraliza a lógica principal, gerenciando o estado do jogo, as entradas do jogador, colisões e orquestrando as demais entidades

4. Implementação dos conceitos de Computação Gráfica

4.1 Pipeline Fixo e Renderização

No projeto a renderização de todas as entidades foi realizada utilizando o pipeline fixo (legacy) do OpenGL. Isso envolve o uso de comandos *glBegin(PRIMITIVA)* e *glEnd()* para definir as geometrias:

- Triângulos (*GL_TRIANGLES*): Usados para desenhar o Player (canhão).
- Quadriláteros (*GL_QUADS*): Usados para desenhar os Projectile e os Alien.

- Pontos (*GL_POINTS*): Usados para criar um efeito de estrelas no fundo da cena.

4.2 Projeção Ortográfica 2D

Na parte de projeção, mapeamos o sistema de coordenadas do mundo 2D para a tela utilizando a função *gluOrtho2D(0, width, 0, height)*. Essa função, chamada na callback de *reshape* (redimensionamento da janela), define uma projeção ortográfica onde as coordenadas dos objetos correspondem diretamente às coordenadas de pixels da tela, simplificando o posicionamento das entidades.

4.3 Loop de Jogo e Animação

Diferente de uma cena estática, um jogo requer atualização constante. Isso foi implementado através da função *glutTimerFunc*, que registra uma callback para ser chamada após um determinado número de milissegundos. Ao configurar o timer para aproximadamente 16ms (1000ms / 60 FPS) e registrar a si mesma recursivamente, cria-se um *game loop* estável para atualizações de lógica e redesenho da tela.

4.4 HUD e Texto Bitmap

Para a exibição do HUD (Score, Lives) e das mensagens de estado (GAME OVER, YOU WIN), foi utilizada a funcionalidade de renderização de texto do GLUT. A função *glutBitmapCharacter* foi usada para desenhar caracteres na tela em uma posição específica, permitindo a comunicação de informações vitais ao jogador.

5. Áudio e Gestão de Estado

Para gerenciar o jogo, na parte mais crucial, foi criado uma máquina de estados finitos para gerenciar o fluxo do jogo:

- **Menu:** Estado inicial, aguardando o início
- **Playing:** Estado principal onde o jogo ocorre
- **Game_Over:** Estados finais que exibe o resultado da partida

6. Conclusão

O desenvolvimento do jogo "Space Invaders" permitiu a aplicação prática e aprofundada dos conceitos teóricos da Computação Gráfica. O projeto cumpriu com sucesso todos os seus objetivos, resultando em um jogo com aplicações interativas e funcionais. A arquitetura do projeto utiliza três bibliotecas distintas, sendo elas o OpenGL, GLUT e SDL2_mixer; demonstra os desafios e as soluções para criar uma experiência em um jogo, combinando renderização gráfica, gerenciamento de eventos e sistemas de áudio; como também o uso do pipeline fixo do OpenGL, provou ser uma ferramenta didática eficaz para solidificar a compreensão das primitivas gráficas e do gerenciamento de matrizes .

7. Git Hub

[Link GitHub - OpenGL 2D](#)